

Flexible Street Platform · 项目交接文档

Flexible Street Platform · 项目交接文档

本文档面向**新接手项目的开发者**，目标是从零开始搭起本地开发环境、理解代码结构、知道部署流程，最终能独立维护与扩展项目。

代码仓库： <https://github.com/FANYANG0304/flexible-street-platform> **报告日期：** 2026-05 **维护者：** （请新接手的人在此填入自己的联系方式）

0. 一分钟概览

这是什么： 一个面向城市规划者的“灵活街道”决策辅助平台。在一张费城地图上，根据 POI 密度 + 实时数据（天气 / 交通 / 节日 / 活动）+ AI 街景视觉评分，给每条街道打一个 0-100 的“灵活使用潜力”分（FSI）。叠加紧急通道一票否决、簇群协调提示、所有权可关闭性等约束。

技术栈： React 18 + TypeScript + Vite（前端）· Mapbox GL JS（地图）· Supabase / PostgreSQL（后端）· Tailwind CSS（样式）· LLaVA（离线 AI 视觉，结果存在 Supabase 表里）

运行形态： 纯静态站点 — 所有评分逻辑在浏览器端实时计算，后端只负责 Supabase 数据存取（4 个 RPC 函数）。无独立服务端进程。

部署： 推送到 `main` 分支后，GitHub Actions 自动构建并发布到 GitHub Pages。

1. 从 GitHub 获取项目代码

1.1 前置工具

工具	最低版本	验证命令
Git	2.30+	<code>git --version</code>
Node.js	18.x 或更高	<code>node --version</code>
npm	9+（随 Node 一起装）	<code>npm --version</code>

Node 18 是必需的——`package.json` 依赖的 React 18、Vite 5、Mapbox GL 3 都要求 Node 18+，而且 GitHub Actions 上跑的也是 Node 18。装更老的版本会在 `npm install` 阶段直接报错。

1.2 克隆仓库

```
git clone https://github.com/FANYANG0304/flexible-street-platform.git
cd flexible-street-platform
```

1.3 安装依赖

```
npm install
```

第一次安装会拉 ~600MB 的 `node_modules`，需要 1-3 分钟。如果国内网络慢，可以先切到淘宝镜像：

```
npm config set registry https://registry.npmmirror.com
npm install
```

2. 配置环境变量（这是最容易踩坑的一步）

2.1 为什么从 GitHub clone 下来运行不起来？

关键原因： 仓库里没有任何 **API key**。从 GitHub clone 下来的代码是不完整的——所有敏感信息（Mapbox token、Supabase 密钥、Google API key 等）都通过 `.env` 文件管理，而 `.env` 被 `.gitignore` 显式排除（`.gitignore` 的第 15、42 行）。

这是**故意的、也是必须的**——把 API key 提交到公开仓库等于把家门钥匙挂在街上，会被别人扫描盗用，付费 API 还会刷你账单。

仓库里你能看到的相关文件只有： - `.env.example` — **模板文件**，列出需要哪些 key，但所有值都是占位符 - `.gitignore` — 配置 `.env` 不被提交

2.2 配置步骤

```
# 复制模板
cp .env.example .env

# 用编辑器打开 .env，把每个占位符替换成真实值
notepad .env      # Windows
# 或
code .env         # 用 VS Code
```

2.3 5 个环境变量详解

变量名	必需?	用途	申请地址	大致费用
VITE_MAPBOX_TOKEN	✔ 必需	Mapbox 地图渲染、矢量瓦片访问	https://account.mapbox.com/access-tokens/	免费层每月 50,000 次地图加载
VITE_SUPABASE_URL	✔ 必需	Supabase 后端 URL	https://supabase.com/dashboard → Project Settings → API	免费
VITE_SUPABASE_ANON_KEY	✔ 必需	Supabase 公共匿名访问密钥	同上	免费
VITE_GOOGLE_SV_KEY	⚠ 可选	Google Street View 街景图 (详情面板用)	https://console.cloud.google.com/ → 启用 “Street View Static API”	每月 28,000 次免费, 超出按 \$0.007/次计费
VITE_TICKETMASTER_KEY	⚠ 可选	拉取季节性活动 (游行、节日) 作为评分加成	https://developer.ticketmaster.com/	免费层 5,000 次/天

可选变量缺失会怎样? - 没有 `VITE_GOOGLE_SV_KEY` → 详情面板的街景图区域会消失, 其它一切正常。
- 没有 `VITE_TICKETMASTER_KEY` → `events` 修正因子永远为 1.0 (不加成), 其它一切正常。

2.4 最重要的安全提醒

⚠ **所有 `VITE_*` 前缀的变量都会被打包进前端 JS, 最终在浏览器中可见。** 这是 Vite 的设计 —— 凡是要在浏览器里用的变量, 都必须带 `VITE_` 前缀。

这意味着这些 key 本质上是**公开的**。保护方式靠的不是隐藏, 而是在 **API 服务商一侧加访问限制**:

- **Mapbox**: 在 token 设置页加 URL referrer 限制, 只允许你的域名访问
- **Google Cloud**: 在 API key 设置页加 HTTP referrer 限制
- **Ticketmaster**: 免费层有限速, 问题不大
- **Supabase Anon Key**: Supabase 的 Row Level Security (RLS) 是真正的防线——anon key 本身就是设计成可公开的

永远不要把私钥 (如 Supabase service_role key、Anthropic API key 等服务端密钥) 放到 `VITE_*` 变量里。

2.5 GitHub Secrets (仅当部署到 GitHub Pages 时)

如果你要让 GitHub Actions 自动构建并部署 ([.github/workflows/deploy.yml](#)) , 需要在仓库后台配置 4 个 secret:

```
GitHub 仓库 → Settings → Secrets and variables → Actions → New repository secret
```

Secret 名	值
VITE_MAPBOX_TOKEN	你的 Mapbox token
VITE_SUPABASE_URL	你的 Supabase URL
VITE_SUPABASE_ANON_KEY	你的 Supabase anon key
VITE_GOOGLE_SV_KEY	你的 Google SV key

当前 workflow 没有配置 `VITE_TICKETMASTER_KEY` 。如果想在线上也启用活动加成, 需要在 `deploy.yml` 里加一行 `VITE_TICKETMASTER_KEY: ${ secrets.VITE_TICKETMASTER_KEY }` 并在 `secrets` 里建对应条目。

3. Supabase 后端配置

平台所有持久化数据 (POI、街道 AI 分数、Playstreets、Open Streets 活动) 都存在 Supabase。前端只通过 4 个 RPC 函数访问后端, 没有任何直接的 `supabase.from(table)` 表操作。

3.1 RPC 函数清单

RPC 名	参数	用途	调用位置
get_anchors_in_bounds	min_lat, min_lng, max_lat, max_lng, scenario_ids[]	取边界范围内的 POI 锚点 (按场景过滤)	MapComponent.tsx:299
get_poi_in_bounds	min_lat, min_lng, max_lat, max_lng	取边界范围内的所有 POI (FSI 评分用)	MapPage.tsx:194、MapComponent.tsx:315
get_playstreets_lines_in_bounds	min_lat, min_lng, max_lat, max_lng	取边界范围内的 Playstreets 线 (验证集)	MapPage.tsx:207、MapComponent.tsx:330
get_street_events_in_bounds	min_lat, min_lng, max_lat, max_lng	取边界范围内的 Open Streets 活动	MapPage.tsx:210、MapComponent.tsx:346

此外还有一个表是直接分页加载的： - `street_ai_scores` — LLaVA 离线分析结果，初始化时一次性加载（每批 1000 行）。详见 [streetScores.ts](#)。

3.2 接管 Supabase 项目的两种方式

方式 A：沿用现有 Supabase 项目（推荐）

让前任开发者把你加到 Supabase 项目里：

```
Supabase Dashboard → 你的项目 → Settings → Team → Add member
```

加入后，你就能直接拿到该项目的 `URL` 和 `anon key`，填进 `.env` 就能跑。所有 RPC、表、数据都已就位。

方式 B：自己起一个新 Supabase 项目（如果原项目无法转移）

需要重建以下内容（联系前任开发者拿到 schema 导出文件最快）： 1. 数据库表：`poi`、`street_ai_scores`、`playstreets`、`street_events` 等 2. 4 个 RPC 函数（PL/pgSQL 实现，本质是带空间过滤的 SELECT） 3. 数据导入：POI 从 OpenStreetMap，AI 分数需要重新跑 LLaVA 流程

建议优先走方式 A——重建后端工作量很大（POI 数据从 OSM 抽取并清洗、AI 分数需要离线跑 LLaVA），从零开始预计 1-2 周。

4. 启动开发服务器

```
npm run dev
```

正常输出会是：

```
VITE v5.4.21 ready in 800 ms
→ Local:   http://localhost:5173/
→ Network: use --host to expose
```

打开 `http://localhost:5173/` 应该看到：1. 首页（Landing page）2. 进入 `/map` 后能看到费城地图加载 3. 打开侧边栏的“Flexibility Score”开关后，街道开始上色

4.1 运行后的健康检查清单

现象	含义	排查
地图完全空白	Mapbox token 错误或缺失	检查 <code>.env</code> 里 <code>VITE_MAPBOX_TOKEN</code> ，重启 <code>npm run dev</code>
地图有底图但 POI 不出现	Supabase URL/key 错误，或 RPC 函数未部署	浏览器 F12 看 Network → 找 <code>supabase.co</code> 的请求是 401 还是 404
Flexibility Score 全是灰色	POI 加载成功但 AI/POI 还没开始算	等 1-2 秒，或拉到至少 zoom 14
详情面板缺街景图	Google SV key 缺失或错误	检查 <code>VITE_GOOGLE_SV_KEY</code> ，或忽略（可选项）
控制台报 <code>Missing Supabase env vars</code>	<code>.env</code> 没生效	确认文件名是 <code>.env</code> 不是 <code>env.txt</code> ，重启 dev server

5. 项目代码结构详解

```
flexible-street-platform/
├── .github/workflows/deploy.yml # GitHub Actions 自动部署到 Pages
├── public/ # 静态资源（直接拷贝到 dist 不经构建）
├── src/ # 所有源码
│   ├── components/ # React 组件
│   ├── pages/ # 页面级组件（路由入口）
│   ├── lib/ # 业务逻辑、算法、API 封装
│   ├── data/ # 静态配置数据（场景、锚点类型等）
│   ├── types/ # 共享 TypeScript 类型定义
│   ├── App.tsx # 根组件 + 路由
│   ├── main.tsx # React 入口
│   └── index.css # Tailwind 引入 + 全局样式
├── reports/ # 项目结项报告（PDF/HTML）
├── .env.example # 环境变量模板（仓库内）
├── .env # 真实环境变量（git 忽略，本地手建）
├── .gitignore
├── index.html # Vite 入口 HTML
├── package.json
├── tsconfig*.json
├── tailwind.config.js
├── vite.config.ts
├── DEPLOYMENT.md # 旧的部署指南（Vercel/Netlify/Docker）
├── README.md # 项目介绍
└── HANDOVER.md # 本文档
```

5.1 `src/components/` — UI 组件

文件	作用
<code>MapComponent.tsx</code>	核心组件。 Mapbox 地图初始化、所有图层管理（街道分数、POI、活动、集群、紧急通道 veto）、RPC 数据加载、点击/悬浮交互。约 1300 行，是整个项目最复杂的文件
<code>Sidebar.tsx</code>	左侧栏：时段选择、场景过滤、各类图层 toggle
<code>MapLegend.tsx</code>	右下角图例：分数色阶、所有权颜色、紧急约束等
<code>StreetScorePanel.tsx</code>	点击街道后弹出的详情面板：分数、子分数、AI 视觉、修正因子、紧急 veto 横幅、集群徽章。可拖拽
<code>StreetEventPanel.tsx</code>	点击 Open Streets 事件后的详情面板（含街景图）
<code>AnchorDetailPanel.tsx</code>	点击 POI 锚点的详情面板

5.2 `src/pages/` — 路由页面

文件	作用
<code>LandingPage.tsx</code>	首页 <code>/</code>
<code>MapPage.tsx</code>	主地图页 <code>/map</code> 。负责调用 Supabase 加载 POI、AI 分数、Playstreets、活动；天气/节日数据获取；把这些 props 传给 <code>MapComponent</code>
<code>Dashboard.tsx</code>	数据看板 <code>/dashboard</code> （统计页面）

5.3 `src/lib/` — 业务逻辑（纯函数为主，可单测）

文件	作用
<code>fsiScores.ts</code>	算法核心 。POI 走廊距离衰减、sigmoid 饱和评分、综合分计算、所有权分级、交通修正、紧急通道 veto、簇群检测（union-find）。所有评分相关逻辑都在这一个文件里。约 700 行
<code>fsiCalibrate.ts</code>	用 Playstreets 验证集反向拟合饱和参数的工具，可在浏览器控制台调用
<code>streetScores.ts</code>	LLaVA AI 分数从 Supabase 分页加载并构建查找索引
<code>events.ts</code>	Ticketmaster API 拉取活动 + 费城本地节日日历 + 计算事件邻近修正
<code>weather.ts</code>	Open-Meteo API 拉取实时/预报天气，映射到 0.2-1.0 修正系数
<code>supabase.ts</code>	Supabase 客户端单例（检查 env、初始化）

5.4 `src/data/` 与 `src/types/`

- `data/mockData.ts` — 场景定义（school dismissal、weekend market 等 6 个）、试点区域 bbox、时段 bin
- `types/index.ts` — 跨文件共享的 TypeScript interface（Anchor、Scenario、StreetScore 等）

5.5 关键数据流图

```
用户打开 /map
↓
MapPage 加载初始数据:
├─ Supabase RPC: get_poi_in_bounds (整个费城)
├─ Supabase 分页: street_ai_scores
├─ Supabase RPC: get_playstreets_lines_in_bounds
├─ Supabase RPC: get_street_events_in_bounds
├─ Open-Meteo API: 当下天气
└─ Ticketmaster API: 季节性活动
↓
传 props 给 MapComponent
↓
Mapbox 地图渲染: 街道矢量瓦片 + POI 圆点 + 事件层
↓
用户开启 "Flexibility Score" 开关
↓
MapComponent.applyScores() 触发:
  对视口内每条街道:
    ├─ computePoiFSI(coords, pois) → commercial/community/mobility 子分
    ├─ getEmergencyVeto(coords, pois) → 紧急通道判断
    ├─ getTrafficModifier / getEventModifier / weather / holiday → 修正因子
    └─ computeCompositeTotal(...) → 最终 0-100 综合分
  写入 Mapbox feature-state 触发图层重绘
↓
applyClusters() 后处理:
  对所有 ≥75 分 + 可关闭 + 未否决的街道:
    findClusters(...) → 共享路口的并查集
    ≥3 条相连 → 写入 clusterSize feature-state
  触发青色集群光晕图层
```

6. 构建与部署

6.1 本地构建

```
npm run build
```

执行 `tsc -b && vite build`，先做 TypeScript 类型检查，再用 Vite 打包到 `dist/`。

构建输出大约 2.2MB JS（gzip 后 ~620KB），单文件 bundle。Vite 也会输出 CSS 和静态资源到 `dist/assets/`。

构建产物可以直接静态托管（不需要 Node 服务）。

6.2 自动部署 (推荐)

仓库已配置 GitHub Actions 自动部署 ([.github/workflows/deploy.yml](#))。流程：

```
push 到 main 分支
↓
GitHub Actions 触发
↓
checkout → 装 Node 18 → npm install
↓
npm run build (注入 4 个 secret 作为环境变量)
↓
上传 dist/ 到 GitHub Pages 制品
↓
deploy 到 GitHub Pages
```

前提： 已在仓库 Settings → Pages 启用 GitHub Pages (source 选 GitHub Actions)，并配好 \$2.5 的 4 个 secret。

6.3 其它部署平台

详见原 [DEPLOYMENT.md](#)，其中已涵盖： - Vercel (推荐替代 GitHub Pages，CDN 更好) - Netlify - Docker 容器化 - 自托管 Nginx 配置

唯一要注意的是：**所有平台都要在控制台单独配置 5 个 `VITE_*` 环境变量**，不能依赖 `.env` 文件 (构建时才需要读取)。

7. 常见问题与排查

7.1 `npm install` 卡在某个包

通常是网络问题，切换镜像即可：

```
npm config set registry https://registry.npmmirror.com
rm -rf node_modules package-lock.json
npm install
```

7.2 TypeScript 报错 `Cannot find module ...`

确认 IDE 用的是项目里的 TS 版本 (VS Code 右下角语言指示器 → “Use Workspace Version”)。

7.3 地图加载后过几秒整个页面崩溃

通常是 Mapbox token 过期或被吊销。去 <https://account.mapbox.com/access-tokens/> 检查。

7.4 街道分数图层始终是灰的

按顺序排查：1. 浏览器控制台是否有 `Missing Supabase env vars` 警告？→ 检查 `.env` 2. Network 面板里 supabase.co 的 RPC 请求返回什么？401 = key 错；404 = RPC 函数不存在；500 = SQL 报错 3. 拉到 zoom ≥ 14 （街道图层在低 zoom 是不显示的）

7.5 GitHub Actions 部署失败

最常见原因是缺少 secret。日志里会看到类似 `VITE_MAPBOX_TOKEN is not defined` 的错。补全 secret 后 re-run workflow 即可。

7.6 部署后线上加载比本地慢很多

GitHub Pages 在国内访问速度不稳。考虑迁移到 Vercel（全球 CDN，对国内更友好）。

8. 后续开发的常用模式

8.1 添加一个新的实时调节因子

例如要加“空气质量”修正：

1. 在 `src/lib/` 新建 `airquality.ts`，写 `fetchAirQuality()` 和 `getAirQualityModifier()`
2. 在 `MapPage.tsx` 加载后传给 `MapComponent`
3. 在 `fsiScores.ts` 的 `computeCompositeTotal()` 函数签名加一个 `airMod` 参数
4. 在 `MapComponent.tsx` `applyScores()` 调用处传入
5. 在 `StreetScorePanel.tsx` 的因子列表里加一项

8.2 添加一个新的地图叠 layers

参考现有 `street-veto-blocked` 或 `street-cluster-outline` 层的实现：1. 在 `MapComponent` 的图层初始化区添加 `map.current.addLayer({...})` 2. 在 visibility toggle 区把新 id 加入 layer ID 数组 3. 在 closeable-only 过滤区也对应加 `setFilter` 调用 4. 如需用 feature-state 驱动样式，定义对应的 `['feature-state', 'xxx']` 表达式常量

8.3 修改评分参数

直接编辑 `src/lib/fsiScores.ts` 顶部的常量：- `CORRIDOR_WIDTH_M` — 各维度走廊宽度 - `DECAY_PERP_M` — 距离衰减系数 - `SATURATION` — sigmoid 饱和参数（影响整体分数分布） - `PROMINENCE_BONUS` — 邻域突出度奖励上限

修改后建议在浏览器控制台跑 `logCalibration()` 验证 Playstreets 正例是否仍能落在 ≥ 75 分区间。

8.4 修改 Supabase RPC

不要改 RPC 函数签名 —— 前端代码硬编码了参数名。如果一定要改： 1. 先在 Supabase 仪表板的 SQL Editor 重写 RPC 2. 同步更新 `MapComponent.tsx`、`MapPage.tsx` 对应的 `supabase.rpc(...)` 调用

9. 关键资源链接

资源	链接
代码仓库	https://github.com/FANYANG0304/flexible-street-platform
在线 Demo (GitHub Pages)	https://fanyang0304.github.io/flexible-street-platform/ (如果已部署)
Mapbox 文档	https://docs.mapbox.com/mapbox-gl-js/api/
Supabase 文档	https://supabase.com/docs
Vite 文档	https://vitejs.dev/
项目结项报告	reports/final-report.pdf
旧部署指南	DEPLOYMENT.md
项目说明	README.md

10. 交接 Checklist

新接手者建议按以下清单确认所有交接事项已到位：

- ☐ 已被加入 GitHub 仓库 (拥有 push 权限)
- ☐ 已被加入 Supabase 项目 (能看到 dashboard 和 keys)
- ☐ 已拿到 Mapbox account 访问 (或申请新 token)
- ☐ 已拿到 Google Cloud project 访问 (或申请新 SV key)
- ☐ 已拿到 Ticketmaster developer 账号 (或申请新 key)
- ☐ 本地 `npm run dev` 能正常启动并在 localhost:5173 看到地图
- ☐ 本地能看到街道分数色块、点击街道弹出详情面板
- ☐ 本地能看到紧急通道 veto (粉红色)
- ☐ 本地能看到族群光晕 (青色)
- ☐ `npm run build` 成功, dist/ 目录正常生成

- ☐ 已读完 [reports/final-report.pdf](#) 了解项目背景与设计决策
- ☐ 已读完 [src/lib/fsiScores.ts](#) 头部注释了解算法
- ☐ GitHub repo Settings → Secrets 里 4 个 secret 都已配好（如果继续用 GitHub Pages）

完成以上所有项后，你就完整掌握这个项目了。祝顺利！

如有疑问，请先查看本文档对应章节，再翻 [reports/final-report.pdf](#)，最后联系前任维护者。